



VIT[®]
BHOPAL
www.vitbhopal.ac.in

Laboratory Record

MAS5001 Artificial Intelligence

Slot: C14 + E11 + E12

Class Number: BL2026270100787

Fall Semester: 2026-2027

Venue: AB02-201

Name of the student: Shiva Kumar

Register Number: 26MAS10029

Index

Exp. No.	Name of the Experiment	Page Number
1.	Implementing Depth-First Search for traversing a graph with 20 nodes.	3
2.	Implementing Breadth-First Search for traversing a graph with 20 nodes.	6
3.	Building an 8-puzzle solver using A-star search algorithm with 10 nodes.	10
4.	Implementing Uniform Cost Search for traversing a graph with 20 nodes.	14
5.	Implementing Depth-Limited Search for traversing a graph with 25 nodes.	17
6.	Implementing Iterative Deepening DFS for traversing a graph with 25 nodes.	21
7.	Implementing Greedy Breadth First Search for traversing a graph with 25 nodes.	24
8.	Implementing A* Search algorithm for traversing a graph with 25 nodes.	28
9.	Write a Python program of hill climbing search with 20 nodes.	32
10.	Write a Python program of beam search algorithm with 20 nodes.	35

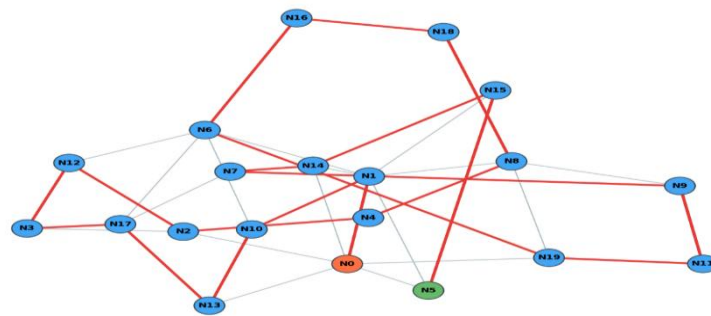
Exp No: 1	Implementing Depth-First Search for traversing a graph with 20 nodes.
26-07-2026	

Aim:

To design and implement the Depth-First Search (DFS) algorithm for traversing a graph containing 20 nodes.

Objective:

1. To write a Python program to implement the Depth-First Search (DFS) algorithm.
2. To traverse a graph consisting of 20 nodes using DFS.
3. To understand the working of stack-based graph traversal and visited node tracking.



Program Code:

```
import random
def build_graph(num_nodes=20, extra_edges=15, seed=42):
    """Builds a random connected undirected graph with `num_nodes`
    nodes."""
    random.seed(seed)
    nodes = [f"N{i}" for i in range(num_nodes)]
    graph = {node: [] for node in nodes}
    # Ensure connectivity: link each node to a random earlier node
    for i in range(1, num_nodes):
        j = random.randint(0, i - 1)
        u, v = nodes[i], nodes[j]
        graph[u].append(v)
        graph[v].append(u)
    # Add a few extra random edges for a more interesting graph
```

```

for _ in range(extra_edges):
    u, v = random.sample(nodes, 2)
    if v not in graph[u]:
        graph[u].append(v)
        graph[v].append(u)

# Keep neighbour lists sorted for deterministic traversal order
for node in graph:
    graph[node] = sorted(set(graph[node]))
return graph

def dfs(graph, start):
    """Iterative DFS traversal (stack-based) returning visit order."""
    visited = set()
    order = []
    stack = [start]
    while stack:
        node = stack.pop()
        if node not in visited:
            visited.add(node)
            order.append(node)
            # Push neighbours in reverse so smaller labels are explored
            first
            for neighbour in reversed(graph[node]):
                if neighbour not in visited:
                    stack.append(neighbour)

    return order

def dfs_recursive(graph, start, visited=None, order=None):
    """Recursive DFS traversal (alternative implementation)."""
    if visited is None:
        visited = set()
        order = []
    visited.add(start)
    order.append(start)
    for neighbour in graph[start]:
        if neighbour not in visited:
            dfs_recursive(graph, neighbour, visited, order)
    return order

def print_graph(graph):
    print("Graph adjacency list:")
    for node, neighbours in graph.items():
        print(f" {node}: {neighbours}")
    print()

if __name__ == "__main__":
    NUM_NODES = 20

```

```

graph = build_graph(NUM_NODES)
print_graph(graph)
start_node = "N0"
print(f"DFS traversal (iterative) starting from {start_node}:")
result_iterative = dfs(graph, start_node)
print(" -> ".join(result_iterative))
print(f"Total nodes visited: {len(result_iterative)}\n")
print(f"DFS traversal (recursive) starting from {start_node}:")
result_recursive = dfs_recursive(graph, start_node)
print(" -> ".join(result_recursive))
print(f"Total nodes visited: {len(result_recursive)}")

```

Output:

Graph adjacency list:

```

N0: ['N1', 'N13', 'N14', 'N19', 'N2', 'N5']
N1: ['N0', 'N10', 'N14', 'N15', 'N5', 'N6', 'N7', 'N8']
N2: ['N0', 'N12', 'N3', 'N4']
N3: ['N12', 'N17', 'N2']
N4: ['N2', 'N8']
N5: ['N0', 'N1']
N6: ['N1', 'N10', 'N12', 'N16', 'N17']
N7: ['N1', 'N14', 'N17']
N8: ['N1', 'N18', 'N19', 'N4', 'N9']
N9: ['N11', 'N8']
N10: ['N1', 'N13', 'N6']
N11: ['N19', 'N9']
N12: ['N2', 'N3', 'N6']
N13: ['N0', 'N10', 'N17']
N14: ['N0', 'N1', 'N7']
N15: ['N1']
N16: ['N18', 'N6']
N17: ['N13', 'N3', 'N6', 'N7']
N18: ['N16', 'N8']
N19: ['N0', 'N11', 'N8']

```

```

DFS traversal (iterative) starting from N0:
N0 -> N1 -> N10 -> N13 -> N17 -> N3 -> N12 -> N2 -> N4 -> N8 -> N18 -> N16 -> N6 ->
N19 -> N11 -> N9 -> N7 -> N14 -> N15 -> N5
Total nodes visited: 20

```

```

DFS traversal (recursive) starting from N0:
N0 -> N1 -> N10 -> N13 -> N17 -> N3 -> N12 -> N2 -> N4 -> N8 -> N18 -> N16 -> N6 ->
N19 -> N11 -> N9 -> N7 -> N14 -> N15 -> N5
Total nodes visited: 20

```

Colab Link:

<https://colab.research.google.com/drive/15B4pt5DSIZpnaRfnUVzq63b44z8R93GC?usp=sharing>

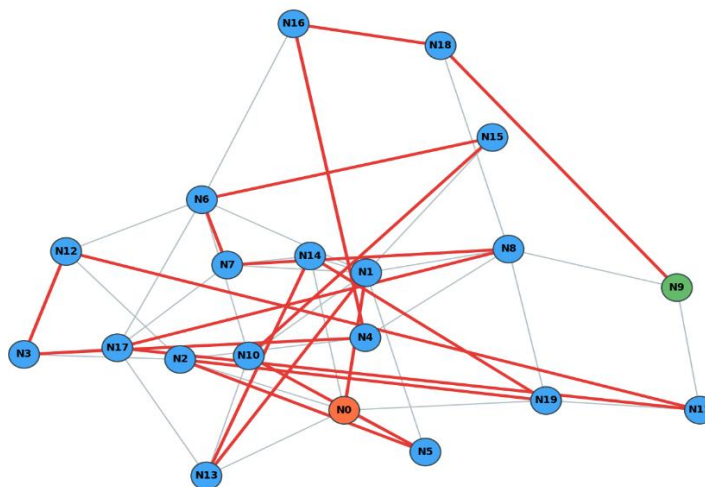
Exp No: 2	Implementing Breadth-First Search for traversing a graph with 20 nodes.
26-07-2026	

Aim:

To design and implement the Breadth-First Search (BFS) algorithm for traversing a graph containing 20 nodes.

Objective:

1. To write a Python program to implement the Breadth-First Search (BFS) algorithm.
2. To traverse a graph consisting of 20 nodes using BFS.
3. To understand queue-based graph traversal and level-wise exploration.



Algorithm :

```
import random
from collections import deque
def build_graph(num_nodes=20, extra_edges=15, seed=42):
    """Builds a random connected undirected graph with `num_nodes` nodes."""
    random.seed(seed)
    nodes = [f"N{i}" for i in range(num_nodes)]
    graph = {node: [] for node in nodes}
    for i in range(1, num_nodes):
        j = random.randint(0, i - 1)
        u, v = nodes[i], nodes[j]
```

```

        graph[u].append(v)
        graph[v].append(u)
    for _ in range(extra_edges):
        u, v = random.sample(nodes, 2)
        if v not in graph[u]:
            graph[u].append(v)
            graph[v].append(u)
    for node in graph:
        graph[node] = sorted(set(graph[node]))
    return graph

def bfs(graph, start):
    """BFS traversal using a queue, returns the visit order."""
    visited = {start}
    order = []
    queue = deque([start])
    while queue:
        node = queue.popleft()
        order.append(node)
        for neighbour in graph[node]:
            if neighbour not in visited:
                visited.add(neighbour)
                queue.append(neighbour)
    return order

def bfs_with_levels(graph, start):
    """BFS that also records the level/depth of each node from start."""
    visited = {start}
    level = {start: 0}
    order = []
    queue = deque([start])
    while queue:
        node = queue.popleft()
        order.append(node)
        for neighbour in graph[node]:
            if neighbour not in visited:
                visited.add(neighbour)
                level[neighbour] = level[node] + 1
                queue.append(neighbour)
    return order, level

def print_graph(graph):
    print("Graph adjacency list:")
    for node, neighbours in graph.items():
        print(f" {node}: {neighbours}")
    print()

```

```

if __name__ == "__main__":
    NUM_NODES = 20
    graph = build_graph(NUM_NODES)
    print_graph(graph)
    start_node = "N0"
    print(f"BFS traversal starting from {start_node}:")
    order, levels = bfs_with_levels(graph, start_node)
    print(" -> ".join(order))
    print(f"Total nodes visited: {len(order)}\n")
    print("Level of each node (distance in edges from start):")
    for node in sorted(levels, key=lambda n: (levels[n], n)):
        print(f"    {node}: level {levels[node]}")

```

Output:

Graph adjacency list:

```

N0: ['N1', 'N13', 'N14', 'N19', 'N2', 'N5']
N1: ['N0', 'N10', 'N14', 'N15', 'N5', 'N6', 'N7', 'N8']
N2: ['N0', 'N12', 'N3', 'N4']
N3: ['N12', 'N17', 'N2']
N4: ['N2', 'N8']
N5: ['N0', 'N1']
N6: ['N1', 'N10', 'N12', 'N16', 'N17']
N7: ['N1', 'N14', 'N17']
N8: ['N1', 'N18', 'N19', 'N4', 'N9']
N9: ['N11', 'N8']
N10: ['N1', 'N13', 'N6']
N11: ['N19', 'N9']
N12: ['N2', 'N3', 'N6']
N13: ['N0', 'N10', 'N17']
N14: ['N0', 'N1', 'N7']
N15: ['N1']
N16: ['N18', 'N6']
N17: ['N13', 'N3', 'N6', 'N7']
N18: ['N16', 'N8']
N19: ['N0', 'N11', 'N8']

```

BFS traversal starting from N0:

```

N0 -> N1 -> N13 -> N14 -> N19 -> N2 -> N5 -> N10 -> N15 -> N6 -> N7 -> N8 -> N17 ->
N11 -> N12 -> N3 -> N4 -> N16 -> N18 -> N9

```

Total nodes visited: 20

Level of each node (distance in edges from start):

```

N0: level 0
N1: level 1
N13: level 1
N14: level 1
N19: level 1
N2: level 1
N5: level 1
N10: level 2
N11: level 2
N12: level 2
N15: level 2

```



```
N17: level 2  
N3: level 2  
N4: level 2  
N6: level 2  
N7: level 2  
N8: level 2  
N16: level 3  
N18: level 3  
N9: level 3
```

Colab Link:

https://colab.research.google.com/drive/1LErYdbfsQl_PQxL4e-hk40Q3rOZehsyf?usp=sharing

Exp No: 3	Building an 8-puzzle solver using A-star search algorithm with 10 nodes.
26-07-2026	

Aim :

To design and implement an 8-puzzle solver using the A* Search algorithm.

Objective :

1. To write a Python program to solve the 8-puzzle problem using the A* Search algorithm.
2. To apply heuristic functions for finding the optimal solution path.
3. To analyze the efficiency of the A* Search algorithm in solving state-space problems.

1	2	3
4		6
7	5	8

1	2	3
4	5	6
7		8

1	2	3
4	5	6
7	8	

Algorithm :

```
import heapq
GOAL_STATE = (1, 2, 3, 4, 5, 6, 7, 8, 0)
def manhattan_distance(state, goal=GOAL_STATE):
    """Heuristic: sum of Manhattan distances of each tile from its goal position."""
    distance = 0
    for index, tile in enumerate(state):
        if tile == 0:
            continue
        goal_index = goal.index(tile)
        row1, col1 = divmod(index, 3)
        row2, col2 = divmod(goal_index, 3)
        distance += abs(row1 - row2) + abs(col1 - col2)
    return distance
def get_neighbours(state):
    """Generates all valid states reachable by sliding a tile into the blank."""
    neighbours = []
```

```

blank_index = state.index(0)
row, col = divmod(blank_index, 3)
moves = {
    "Up": (row - 1, col),
    "Down": (row + 1, col),
    "Left": (row, col - 1),
    "Right": (row, col + 1),
}
for move, (new_row, new_col) in moves.items():
    if 0 <= new_row < 3 and 0 <= new_col < 3:
        new_index = new_row * 3 + new_col
        new_state = list(state)
        new_state[blank_index], new_state[new_index] = (
            new_state[new_index],
            new_state[blank_index],
        )
        neighbours.append((move, tuple(new_state)))
return neighbours

def a_star_solve(start_state, goal_state=GOAL_STATE):
    """Solves the 8-puzzle using A* search (f = g + h)."""
    counter = 0 # tie-breaker for heap comparisons
    open_list = [(manhattan_distance(start_state, goal_state), counter, start_state,
[[]])]
    g_scores = {start_state: 0}
    visited = set()
    nodes_expanded = 0
    while open_list:
        f, _, current_state, path = heapq.heappop(open_list)
        if current_state in visited:
            continue
        visited.add(current_state)
        nodes_expanded += 1
        if current_state == goal_state:
            return path, nodes_expanded
        for move, neighbour in get_neighbours(current_state):
            tentative_g = g_scores[current_state] + 1
            if neighbour not in g_scores or tentative_g < g_scores[neighbour]:
                g_scores[neighbour] = tentative_g
                f_score = tentative_g + manhattan_distance(neighbour, goal_state)
                counter += 1
                heapq.heappush(
                    open_list, (f_score, counter, neighbour, path + [(move,
neighbour)]))

```

```

    )
    return None, nodes_expanded # No solution found
def print_state(state):
    for row in range(3):
        print(state[row * 3: row * 3 + 3])
    print()
def is_solvable(state):
    """Checks solvability via inversion count (ignoring the blank tile)."""
    flat = [tile for tile in state if tile != 0]
    inversions = 0
    for i in range(len(flat)):
        for j in range(i + 1, len(flat)):
            if flat[i] > flat[j]:
                inversions += 1
    return inversions % 2 == 0
if __name__ == "__main__":
    # A solvable starting configuration (0 = blank)
    start_state = (1, 2, 3, 4, 0, 6, 7, 5, 8)
    print("Start state:")
    print_state(start_state)
    print("Goal state:")
    print_state(GOAL_STATE)
    if not is_solvable(start_state):
        print("This puzzle configuration is NOT solvable.")
    else:
        solution, expanded = a_star_solve(start_state)
        if solution is None:
            print("No solution found.")
        else:
            print(f"Solved in {len(solution)} moves, expanding {expanded} nodes.\n")
            current = start_state
            for step, (move, state) in enumerate(solution, start=1):
                print(f"Step {step}: move {move}")
                print_state(state)

```

Output :

```

Start state:
(1, 2, 3)
(4, 0, 6)
(7, 5, 8)

```

```

Goal state:
(1, 2, 3)
(4, 5, 6)

```

```
(7, 8, 0)
```

```
Solved in 2 moves, expanding 3 nodes.
```

```
Step 1: move Down
```

```
(1, 2, 3)
```

```
(4, 5, 6)
```

```
(7, 0, 8)
```

```
Step 2: move Right
```

```
(1, 2, 3)
```

```
(4, 5, 6)
```

```
(7, 8, 0)
```

Colab Link :

<https://colab.research.google.com/drive/10BN4PAgPb2IH0aPcLPxKBM44rBna0Lm?usp=sharing>

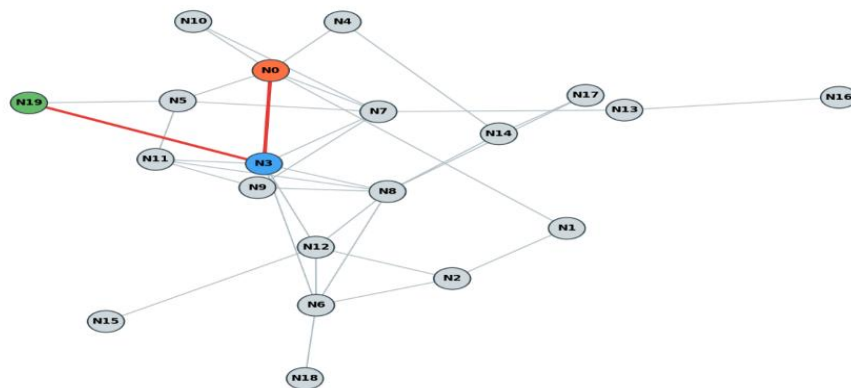
Exp No: 4	Implementing Uniform Cost Search for traversing a graph with 20 nodes.
26-07-2026	

Aim:

To design and implement the Uniform Cost Search (UCS) algorithm for traversing a graph containing 20 nodes.

Objective:

1. To write a Python program to implement the Uniform Cost Search algorithm.
2. To traverse a weighted graph consisting of 20 nodes based on the least path cost.
3. To understand the concept of optimal pathfinding using cumulative path costs.



Program Code:

```

import random
import heapq
def build_weighted_graph(num_nodes=20, extra_edges=15, seed=42,
max_weight=10):
    """Builds a random connected weighted undirected graph."""
    random.seed(seed)
    nodes = [f"N{i}" for i in range(num_nodes)]
    graph = {node: [] for node in nodes}
    for i in range(1, num_nodes):
        j = random.randint(0, i - 1)
        u, v = nodes[i], nodes[j]
        weight = random.randint(1, max_weight)

```

```

graph[u].append((v, weight))
graph[v].append((u, weight))
for _ in range(extra_edges):
    u, v = random.sample(nodes, 2)
    if all(neighbour != v for neighbour, _ in graph[u]):
        weight = random.randint(1, max_weight)
        graph[u].append((v, weight))
        graph[v].append((u, weight))
for node in graph:
    graph[node] = sorted(set(graph[node]))
return graph

def uniform_cost_search(graph, start, goal):
    """UCS: expands the lowest cumulative-cost node first using a priority
    queue."""
    counter = 0
    frontier = [(0, counter, start, [start])]
    visited_cost = {start: 0}
    nodes_expanded = 0
    while frontier:
        cost, _, node, path = heapq.heappop(frontier)
        nodes_expanded += 1
        if node == goal:
            return path, cost, nodes_expanded
        for neighbour, weight in graph[node]:
            new_cost = cost + weight
            if neighbour not in visited_cost or new_cost <
visited_cost[neighbour]:
                visited_cost[neighbour] = new_cost
                counter += 1
                heapq.heappush(frontier, (new_cost, counter, neighbour,
path + [neighbour]))
        return None, float("inf"), nodes_expanded
def print_graph(graph):
    print("Weighted graph adjacency list (node: [(neighbour, weight),
...]):")
    for node, neighbours in graph.items():
        print(f" {node}: {neighbours}")
    print()
if __name__ == "__main__":
    NUM_NODES = 20
    graph = build_weighted_graph(NUM_NODES)
    print_graph(graph)
    start_node, goal_node = "N0", "N19"

```

```

    print(f"Uniform Cost Search from {start_node} to {goal_node}:")
    path, cost, expanded = uniform_cost_search(graph, start_node,
goal_node)
    if path is None:
        print("No path found.")
    else:
        print(" -> ".join(path))
        print(f"Total path cost: {cost}")
        print(f"Nodes expanded: {expanded}")

```

Output:

```

Weighted graph adjacency list (node: [(neighbour, weight), ...]):
N0: [(('N1', 1), ('N10', 9), ('N3', 3), ('N4', 9), ('N5', 10), ('N7', 2))]
N1: [(('N0', 1), ('N2', 4))]
N2: [(('N1', 4), ('N12', 9), ('N6', 1))]
N3: [(('N0', 3), ('N11', 9), ('N12', 6), ('N19', 2), ('N6', 1), ('N7', 7), ('N8',
4))]
N4: [(('N0', 9), ('N14', 1))]
N5: [(('N0', 10), ('N11', 6), ('N19', 9), ('N7', 8))]
N6: [(('N12', 4), ('N18', 6), ('N2', 1), ('N3', 1), ('N8', 2))]
N7: [(('N0', 2), ('N10', 1), ('N13', 10), ('N3', 7), ('N5', 8), ('N9', 2))]
N8: [(('N11', 1), ('N12', 9), ('N14', 6), ('N17', 3), ('N3', 4), ('N6', 2),
('N9', 10))]
N9: [(('N11', 10), ('N7', 2), ('N8', 10))]
N10: [(('N0', 9), ('N7', 1))]
N11: [(('N3', 9), ('N5', 6), ('N8', 1), ('N9', 10))]
N12: [(('N15', 3), ('N2', 9), ('N3', 6), ('N6', 4), ('N8', 9))]
N13: [(('N16', 6), ('N7', 10))]
N14: [(('N17', 2), ('N4', 1), ('N8', 6))]
N15: [(('N12', 3))]
N16: [(('N13', 6))]
N17: [(('N14', 2), ('N8', 3))]
N18: [(('N6', 6))]
N19: [(('N3', 2), ('N5', 9))]

```

```
Uniform Cost Search from N0 to N19:
```

```

N0 -> N3 -> N19
Total path cost: 5
Nodes expanded: 9

```

Colab Link :

<https://colab.research.google.com/drive/1BPfTquZOO7r6CHbw24aCaNQCUQp3StPm?usp=sharing>

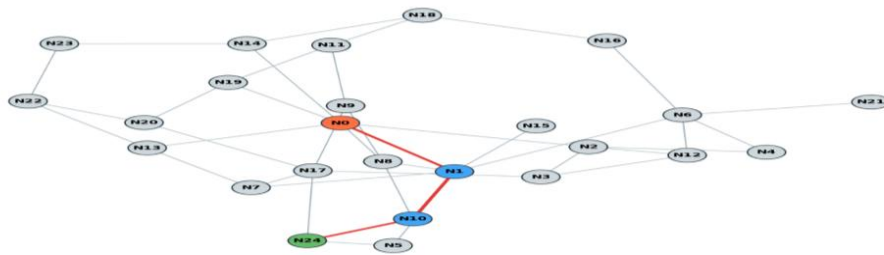
Exp No: 5	Implementing Depth-Limited Search for traversing a graph with 25 nodes.
26-07-2026	

Aim:

To design and implement the Depth-Limited Search (DLS) algorithm for traversing a graph containing 25 nodes.

Objectives:

1. To write a Python program to implement the Depth-Limited Search algorithm.
2. To traverse a graph of 25 nodes with a predefined depth limit.
3. To understand the effect of depth limits on graph traversal.



Program Code:

```
import random
def build_graph(num_nodes=25, extra_edges=18, seed=42):
    """Builds a random connected undirected graph with `num_nodes` nodes."""
    random.seed(seed)
    nodes = [f"N{i}" for i in range(num_nodes)]
    graph = {node: [] for node in nodes}
    for i in range(1, num_nodes):
        j = random.randint(0, i - 1)
        u, v = nodes[i], nodes[j]
        graph[u].append(v)
        graph[v].append(u)
    for _ in range(extra_edges):
        u, v = random.sample(nodes, 2)
        if v not in graph[u]:
            graph[u].append(v)
            graph[v].append(u)
```

```

    for node in graph:
        graph[node] = sorted(set(graph[node]))
    return graph
def depth_limited_search(graph, start, goal, limit):
    """
    Depth-Limited Search. Returns (path, status) where status is one of:
    'found', 'cutoff' (goal might exist beyond the limit), or 'failure'.
    """
    def recursive_dls(node, goal, limit, path, visited):
        if node == goal:
            return path, "found"
        if limit == 0:
            return None, "cutoff"
        cutoff_occurred = False
        for neighbour in graph[node]:
            if neighbour not in visited:
                visited.add(neighbour)
                result_path, status = recursive_dls(
                    neighbour, goal, limit - 1, path + [neighbour], visited
                )
                if status == "found":
                    return result_path, "found"
                if status == "cutoff":
                    cutoff_occurred = True
                visited.discard(neighbour) # allow revisits on other branches
        return None, "cutoff" if cutoff_occurred else "failure"
    visited = {start}
    return recursive_dls(start, goal, limit, [start], visited)
def print_graph(graph):
    print("Graph adjacency list:")
    for node, neighbours in graph.items():
        print(f" {node}: {neighbours}")
    print()
if __name__ == "__main__":
    NUM_NODES = 25
    graph = build_graph(NUM_NODES)
    print_graph(graph)
    start_node, goal_node = "N0", "N24"
    depth_limit = 5
    print(f"Depth-Limited Search from {start_node} to {goal_node} with
limit={depth_limit}:")
    path, status = depth_limited_search(graph, start_node, goal_node, depth_limit)
    if status == "found":

```

```

    print("Goal found!")
    print(" -> ".join(path))
    print(f"Path length: {len(path) - 1} edges")
elif status == "cutoff":
    print(f"Cutoff: goal not found within depth limit {depth_limit} "
          f"(it may exist deeper).")
else:
    print("Failure: goal does not exist in the graph.")

```

Output:

```

Graph adjacency list:
N0: ['N1', 'N13', 'N14', 'N19', 'N2', 'N8']
N1: ['N0', 'N10', 'N15', 'N5', 'N6', 'N7', 'N8']
N2: ['N0', 'N12', 'N3', 'N4']
N3: ['N12', 'N17', 'N2']
N4: ['N2', 'N6']
N5: ['N1', 'N24']
N6: ['N1', 'N12', 'N16', 'N21', 'N4']
N7: ['N1', 'N13', 'N17']
N8: ['N0', 'N1', 'N10', 'N9']
N9: ['N11', 'N17', 'N8']
N10: ['N1', 'N24', 'N8']
N11: ['N18', 'N19', 'N9']
N12: ['N2', 'N3', 'N6']
N13: ['N0', 'N22', 'N7']
N14: ['N0', 'N18', 'N23']
N15: ['N1']
N16: ['N18', 'N6']
N17: ['N20', 'N24', 'N3', 'N7', 'N9']
N18: ['N11', 'N14', 'N16']
N19: ['N0', 'N11', 'N20']
N20: ['N17', 'N19', 'N22']
N21: ['N6']
N22: ['N13', 'N20', 'N23']
N23: ['N14', 'N22']
N24: ['N10', 'N17', 'N5']

```

```

Depth-Limited Search from N0 to N24 with limit=5:
Goal found!
N0 -> N1 -> N10 -> N24
Path length: 3 edges

```

Colab Link :

<https://colab.research.google.com/drive/1C96kFtRvX5RTjxbm9Covnl3JpOPBXWx2?usp=sharing>

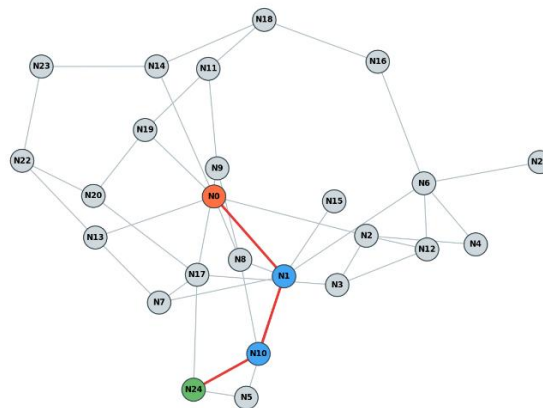
Exp No: 6	Implementing Iterative Deepening DFS for traversing a graph with 25 nodes.
26-07-2026	

Aim:

To design and implement the Iterative Deepening Depth-First Search (IDDFS) algorithm for traversing a graph containing 25 nodes.

Objectives:

1. To write a Python program to implement the IDDFS algorithm.
2. To traverse a graph of 25 nodes using iterative depth limits.
3. To compare the advantages of IDDFS over DFS and DLS.



Program Code:

```
import random
def build_graph(num_nodes=25, extra_edges=18, seed=42):
    """Builds a random connected undirected graph with `num_nodes` nodes."""
    random.seed(seed)
    nodes = [f"N{i}" for i in range(num_nodes)]
    graph = {node: [] for node in nodes}
    for i in range(1, num_nodes):
        j = random.randint(0, i - 1)
        u, v = nodes[i], nodes[j]
        graph[u].append(v)
```

```

        graph[v].append(u)
    for _ in range(extra_edges):
        u, v = random.sample(nodes, 2)
        if v not in graph[u]:
            graph[u].append(v)
            graph[v].append(u)
    for node in graph:
        graph[node] = sorted(set(graph[node]))
    return graph

def depth_limited_search(graph, node, goal, limit, path, visited):
    """Helper: DLS used internally by IDDFS. Returns (path, found)."""
    if node == goal:
        return path, True
    if limit <= 0:
        return None, False
    for neighbour in graph[node]:
        if neighbour not in visited:
            visited.add(neighbour)
            result_path, found = depth_limited_search(
                graph, neighbour, goal, limit - 1, path + [neighbour], visited
            )
            if found:
                return result_path, True
            visited.discard(neighbour)
    return None, False

def iddfs(graph, start, goal, max_depth):
    """Iterative Deepening DFS: repeatedly runs DLS with increasing depth limits."""
    for depth in range(max_depth + 1):
        visited = {start}
        path, found = depth_limited_search(graph, start, goal, depth, [start],
visited)
        print(f" Trying depth limit = {depth} ... "
              f"{'goal found' if found else 'not found'}")
        if found:
            return path, depth
    return None, -1

def print_graph(graph):
    print("Graph adjacency list:")
    for node, neighbours in graph.items():
        print(f" {node}: {neighbours}")
    print()

if __name__ == "__main__":
    NUM_NODES = 25

```

```

graph = build_graph(NUM_NODES)
print_graph(graph)
start_node, goal_node = "N0", "N24"
max_depth = 10
print(f"Iterative Deepening DFS from {start_node} to {goal_node} "
      f"(max depth = {max_depth}):")
path, depth_found = iddfs(graph, start_node, goal_node, max_depth)
print()
if path:
    print(f"Goal found at depth {depth_found}:")
    print(" -> ".join(path))
else:
    print(f"Goal not found within max depth {max_depth}.")

```

Output:

```

Graph adjacency list:
N0: ['N1', 'N13', 'N14', 'N19', 'N2', 'N8']
N1: ['N0', 'N10', 'N15', 'N5', 'N6', 'N7', 'N8']
N2: ['N0', 'N12', 'N3', 'N4']
N3: ['N12', 'N17', 'N2']
N4: ['N2', 'N6']
N5: ['N1', 'N24']
N6: ['N1', 'N12', 'N16', 'N21', 'N4']
N7: ['N1', 'N13', 'N17']
N8: ['N0', 'N1', 'N10', 'N9']
N9: ['N11', 'N17', 'N8']
N10: ['N1', 'N24', 'N8']
N11: ['N18', 'N19', 'N9']
N12: ['N2', 'N3', 'N6']
N13: ['N0', 'N22', 'N7']
N14: ['N0', 'N18', 'N23']
N15: ['N1']
N16: ['N18', 'N6']
N17: ['N20', 'N24', 'N3', 'N7', 'N9']
N18: ['N11', 'N14', 'N16']
N19: ['N0', 'N11', 'N20']
N20: ['N17', 'N19', 'N22']
N21: ['N6']
N22: ['N13', 'N20', 'N23']
N23: ['N14', 'N22']
N24: ['N10', 'N17', 'N5']
Iterative Deepening DFS from N0 to N24 (max depth = 10):
Trying depth limit = 0 ... not found
Trying depth limit = 1 ... not found
Trying depth limit = 2 ... not found
Trying depth limit = 3 ... goal found
Goal found at depth 3:
N0 -> N1 -> N10 -> N24

```

ColabLink:https://colab.research.google.com/drive/1TEvaR_QTk_XEnViiJ0OKslO7SDFtSwy7?usp=sharing

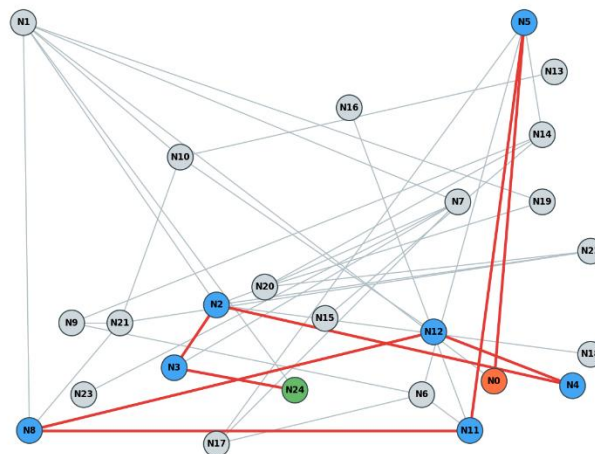
Exp No: 7	Implementing Greedy Breadth First Search for traversing a graph with 25 nodes.
26-07-2026	

Aim:

To design and implement the Greedy Best-First Search algorithm for traversing a graph containing 25 nodes.

Objectives:

1. To write a Python program to implement the Greedy Best-First Search algorithm.
2. To traverse a graph of 25 nodes using heuristic values.
3. To understand heuristic-based search strategies for pathfinding.



Program Code:

```
import random
import heapq
import math
def build_graph_with_coordinates(num_nodes=25, extra_edges=18, seed=42,
grid_size=100):
    """
    Builds a random connected weighted undirected graph where each node has
    (x, y) coordinates. Edge weights are the Euclidean distance between nodes,
```

```

and the heuristic h(n) is the straight-line distance to the goal.
"""

random.seed(seed)
nodes = [f"N{i}" for i in range(num_nodes)]
coordinates = {node: (random.randint(0, grid_size), random.randint(0, grid_size))
               for node in nodes}
graph = {node: [] for node in nodes}
def distance(a, b):
    (x1, y1), (x2, y2) = coordinates[a], coordinates[b]
    return round(math.hypot(x2 - x1, y2 - y1), 2)
for i in range(1, num_nodes):
    j = random.randint(0, i - 1)
    u, v = nodes[i], nodes[j]
    w = distance(u, v)
    graph[u].append((v, w))
    graph[v].append((u, w))
for _ in range(extra_edges):
    u, v = random.sample(nodes, 2)
    if all(neighbour != v for neighbour, _ in graph[u]):
        w = distance(u, v)
        graph[u].append((v, w))
        graph[v].append((u, w))
for node in graph:
    graph[node] = sorted(set(graph[node]))
return graph, coordinates

def heuristic(node, goal, coordinates):
    """Straight-line (Euclidean) distance heuristic."""
    (x1, y1), (x2, y2) = coordinates[node], coordinates[goal]
    return math.hypot(x2 - x1, y2 - y1)

def greedy_best_first_search(graph, start, goal, coordinates):
    """
    Greedy Best-First Search: priority queue ordered only by h(n),
    the estimated distance to the goal.
    """
    counter = 0
    frontier = [(heuristic(start, goal, coordinates), counter, start, [start])]
    visited = set()
    nodes_expanded = 0
    while frontier:
        _, _, node, path = heapq.heappop(frontier)
        if node in visited:
            continue
        visited.add(node)

```



```

        nodes_expanded += 1
    if node == goal:
        return path, nodes_expanded
    for neighbour, _weight in graph[node]:
        if neighbour not in visited:
            counter += 1
            h = heuristic(neighbour, goal, coordinates)
            heapq.heappush(frontier, (h, counter, neighbour, path + [neighbour]))
    return None, nodes_expanded
def print_graph(graph, coordinates):
    print("Graph adjacency list (node: [(neighbour, edge_distance), ...]):")
    for node, neighbours in graph.items():
        print(f"    {node} {coordinates[node]}: {neighbours}")
    print()
if __name__ == "__main__":
    NUM_NODES = 25
    graph, coordinates = build_graph_with_coordinates(NUM_NODES)
    print_graph(graph, coordinates)
    start_node, goal_node = "N0", "N24"
    print(f"Greedy Best-First Search from {start_node} to {goal_node}:")
    path, expanded = greedy_best_first_search(graph, start_node, goal_node,
coordinates)
    if path is None:
        print("No path found.")
    else:
        print(" -> ".join(path))
        print(f"Nodes expanded: {expanded}")

```

Output:

```

Graph adjacency list (node: [(neighbour, edge_distance), ...]):
  N0 (81, 14): [('N1', 111.73), ('N5', 80.16)]
  N1 (3, 94): [('N0', 111.73), ('N10', 39.7), ('N19', 94.85), ('N2',
70.66), ('N24', 93.54), ('N7', 82.37), ('N8', 91.01)]
  N2 (35, 31): [('N1', 70.66), ('N18', 62.97), ('N20', 8.94), ('N22',
63.15), ('N3', 15.65), ('N4', 61.68)]
  N3 (28, 17): [('N2', 15.65), ('N24', 20.62), ('N7', 59.82)]
  N4 (94, 13): [('N12', 25.94), ('N2', 61.68)]
  N5 (86, 94): [('N0', 80.16), ('N11', 91.44), ('N14', 25.18), ('N17',
106.94), ('N6', 84.72)]
  N6 (69, 11): [('N11', 11.31), ('N17', 35.74), ('N5', 84.72), ('N9',
60.17)]
  N7 (75, 54): [('N1', 82.37), ('N17', 67.2), ('N20', 37.22), ('N23',
75.45), ('N3', 59.82)]
  N8 (4, 3): [('N1', 91.01), ('N11', 73.0), ('N12', 70.52), ('N21', 28.3)]

```

```

N9 (11, 27): [('N14', 88.59), ('N21', 8.0), ('N6', 60.17)]
N10 (29, 64): [('N1', 39.7), ('N12', 57.31), ('N13', 64.85), ('N21',
38.33)]
N11 (77, 3): [('N16', 74.73), ('N5', 91.44), ('N6', 11.31), ('N8',
73.0)]
N12 (71, 25): [('N10', 57.31), ('N4', 25.94), ('N8', 70.52)]
N13 (91, 83): [('N10', 64.85)]
N14 (89, 69): [('N15', 54.56), ('N20', 57.2), ('N5', 25.18), ('N9',
88.59)]
N15 (53, 28): [('N14', 54.56)]
N16 (57, 75): [('N11', 74.73)]
N17 (35, 0): [('N5', 106.94), ('N6', 35.74), ('N7', 67.2)]
N18 (97, 20): [('N2', 62.97)]
N19 (89, 54): [('N1', 94.85), ('N20', 49.77)]
N20 (43, 35): [('N14', 57.2), ('N19', 49.77), ('N2', 8.94), ('N22',
54.59), ('N7', 37.22)]
N21 (19, 27): [('N10', 38.33), ('N22', 79.62), ('N8', 28.3), ('N9',
8.0)]
N22 (97, 43): [('N2', 63.15), ('N20', 54.59), ('N21', 79.62)]
N23 (13, 11): [('N7', 75.45)]
N24 (48, 12): [('N1', 93.54), ('N3', 20.62)]

```

Greedy Best-First Search from N0 to N24:

N0 -> N5 -> N11 -> N8 -> N12 -> N4 -> N2 -> N3 -> N24

Nodes expanded: 13

Colab Link :

https://colab.research.google.com/drive/11HmyvJ1fWKY7ycvK_NHzonCqWilPN6tu?usp=sharing

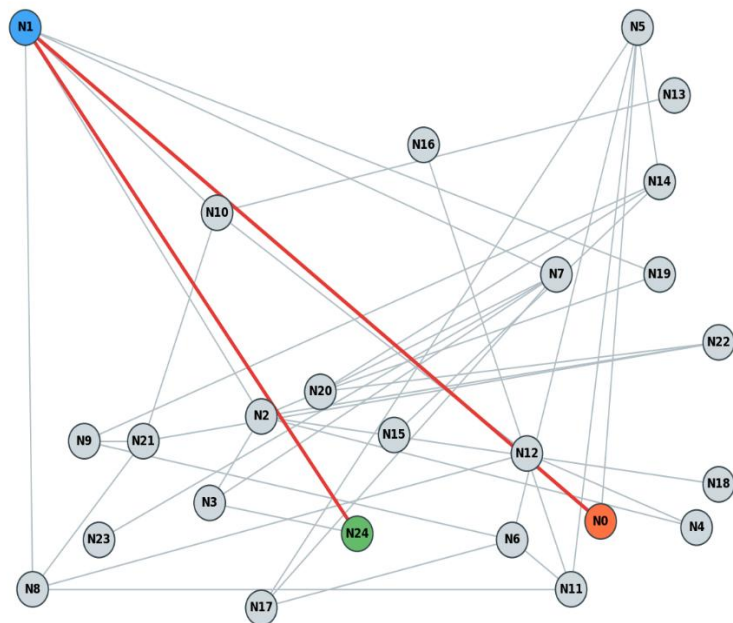
Exp No: 8	Implementing A* Search algorithm for traversing a graph with 25 nodes.
26-07-2026	

Aim:

To design and implement the A* Search algorithm for traversing a graph containing 25 nodes.

Objectives:

1. To write a Python program to implement the A* Search algorithm.
2. To traverse a graph of 25 nodes using path cost and heuristic values.
3. To understand optimal pathfinding using informed search techniques.



Program Code:

```
import random
import heapq
import math

def build_graph_with_coordinates(num_nodes=25, extra_edges=18, seed=42,
grid_size=100):
    """Builds a random connected weighted undirected graph with coordinates."""
    random.seed(seed)
    nodes = [f"N{i}" for i in range(num_nodes)]
    coordinates = {node: (random.randint(0, grid_size), random.randint(0, grid_size))
                     for node in nodes}
    graph = {node: [] for node in nodes}
    def distance(a, b):
        (x1, y1), (x2, y2) = coordinates[a], coordinates[b]
        return round(math.hypot(x2 - x1, y2 - y1), 2)
    for i in range(1, num_nodes):
        j = random.randint(0, i - 1)
        u, v = nodes[i], nodes[j]
        w = distance(u, v)
        graph[u].append((v, w))
        graph[v].append((u, w))
    for _ in range(extra_edges):
        u, v = random.sample(nodes, 2)
        if all(neighbour != v for neighbour, _ in graph[u]):
            w = distance(u, v)
            graph[u].append((v, w))
            graph[v].append((u, w))
    for node in graph:
        graph[node] = sorted(set(graph[node]))
    return graph, coordinates

def heuristic(node, goal, coordinates):
    (x1, y1), (x2, y2) = coordinates[node], coordinates[goal]
    return math.hypot(x2 - x1, y2 - y1)

def a_star_search(graph, start, goal, coordinates):
    """A* search: expands nodes ordered by  $f(n) = g(n) + h(n)$ ."""
    counter = 0
    g_scores = {start: 0}
    f_start = heuristic(start, goal, coordinates)
    frontier = [(f_start, counter, start, [start])]
    visited = set()
    nodes_expanded = 0
    while frontier:
        f, _, node, path = heapq.heappop(frontier)
```

```

        if node in visited:
            continue
        visited.add(node)
        nodes_expanded += 1
        if node == goal:
            return path, g_scores[node], nodes_expanded
        for neighbour, weight in graph[node]:
            tentative_g = g_scores[node] + weight
            if neighbour not in g_scores or tentative_g < g_scores[neighbour]:
                g_scores[neighbour] = tentative_g
                f_score = tentative_g + heuristic(neighbour, goal, coordinates)
                counter += 1
                heapq.heappush(frontier, (f_score, counter, neighbour, path +
[neighbour]))
            return None, float("inf"), nodes_expanded
def print_graph(graph, coordinates):
    print("Graph adjacency list (node: [(neighbour, edge_distance), ...]):")
    for node, neighbours in graph.items():
        print(f"    {node} {coordinates[node]}: {neighbours}")
    print()
if __name__ == "__main__":
    NUM_NODES = 25
    graph, coordinates = build_graph_with_coordinates(NUM_NODES)
    print_graph(graph, coordinates)
    start_node, goal_node = "N0", "N24"
    print(f"A* Search from {start_node} to {goal_node}:")
    path, cost, expanded = a_star_search(graph, start_node, goal_node, coordinates)
    if path is None:
        print("No path found.")
    else:
        print(" -> ".join(path))
        print(f"Total path cost: {round(cost, 2)}")
        print(f"Nodes expanded: {expanded}")

```

Output:

```

Graph adjacency list (node: [(neighbour, edge_distance), ...]):
  N0 (81, 14): [('N1', 111.73), ('N5', 80.16)]
  N1 (3, 94): [('N0', 111.73), ('N10', 39.7), ('N19', 94.85), ('N2', 70.66), ('N24',
93.54), ('N7', 82.37), ('N8', 91.01)]
  N2 (35, 31): [('N1', 70.66), ('N18', 62.97), ('N20', 8.94), ('N22', 63.15), ('N3',
15.65), ('N4', 61.68)]
  N3 (28, 17): [('N2', 15.65), ('N24', 20.62), ('N7', 59.82)]
  N4 (94, 13): [('N12', 25.94), ('N2', 61.68)]
  N5 (86, 94): [('N0', 80.16), ('N11', 91.44), ('N14', 25.18), ('N17', 106.94), ('N6',
84.72)]

```

```

N6 (69, 11): [('N11', 11.31), ('N17', 35.74), ('N5', 84.72), ('N9', 60.17)]
N7 (75, 54): [('N1', 82.37), ('N17', 67.2), ('N20', 37.22), ('N23', 75.45), ('N3',
59.82)]
N8 (4, 3): [('N1', 91.01), ('N11', 73.0), ('N12', 70.52), ('N21', 28.3)]
N9 (11, 27): [('N14', 88.59), ('N21', 8.0), ('N6', 60.17)]
N10 (29, 64): [('N1', 39.7), ('N12', 57.31), ('N13', 64.85), ('N21', 38.33)]
N11 (77, 3): [('N16', 74.73), ('N5', 91.44), ('N6', 11.31), ('N8', 73.0)]
N12 (71, 25): [('N10', 57.31), ('N4', 25.94), ('N8', 70.52)]
N13 (91, 83): [('N10', 64.85)]
N14 (89, 69): [('N15', 54.56), ('N20', 57.2), ('N5', 25.18), ('N9', 88.59)]
N15 (53, 28): [('N14', 54.56)]
N16 (57, 75): [('N11', 74.73)]
N17 (35, 0): [('N5', 106.94), ('N6', 35.74), ('N7', 67.2)]
N18 (97, 20): [('N2', 62.97)]
N19 (89, 54): [('N1', 94.85), ('N20', 49.77)]
N20 (43, 35): [('N14', 57.2), ('N19', 49.77), ('N2', 8.94), ('N22', 54.59), ('N7',
37.22)]
N21 (19, 27): [('N10', 38.33), ('N22', 79.62), ('N8', 28.3), ('N9', 8.0)]
N22 (97, 43): [('N2', 63.15), ('N20', 54.59), ('N21', 79.62)]
N23 (13, 11): [('N7', 75.45)]
N24 (48, 12): [('N1', 93.54), ('N3', 20.62)]

```

A* Search from N0 to N24:

N0 -> N1 -> N24

Total path cost: 205.27

Nodes expanded: 11

Colab Link :

https://colab.research.google.com/drive/1y4rzzr_ZqH5ccd_qGheRLJZYWtvHDjYBC?usp=sharing

Exp No: 9	Write a Python program of hill climbing search with 20 nodes.
26-07-2026	

Aim:

To design and implement the Hill Climbing Search algorithm for solving a graph search problem with 20 nodes.

Objectives:

1. To write a Python program to implement the Hill Climbing Search algorithm.
2. To traverse a graph consisting of 20 nodes using heuristic evaluation.
3. To understand local search techniques and their limitations, such as local maxima.



Program Code:

```
import random
import math
def build_graph_with_coordinates(num_nodes=20, extra_edges=15, seed=42,
grid_size=100):
    """Builds a random connected undirected graph where nodes have coordinates."""
    random.seed(seed)
    nodes = [f"N{i}" for i in range(num_nodes)]
    coordinates = {node: (random.randint(0, grid_size), random.randint(0, grid_size))
                    for node in nodes}
    graph = {node: [] for node in nodes}
    for i in range(1, num_nodes):
        j = random.randint(0, i - 1)
        u, v = nodes[i], nodes[j]
```

```

        graph[u].append(v)
        graph[v].append(u)
    for _ in range(extra_edges):
        u, v = random.sample(nodes, 2)
        if v not in graph[u]:
            graph[u].append(v)
            graph[v].append(u)
    for node in graph:
        graph[node] = sorted(set(graph[node]))
    return graph, coordinates

def heuristic(node, goal, coordinates):
    """Straight-line distance to the goal -- lower is better."""
    (x1, y1), (x2, y2) = coordinates[node], coordinates[goal]
    return math.hypot(x2 - x1, y2 - y1)

def hill_climbing(graph, start, goal, coordinates):
    """
    Simple (greedy) Hill Climbing.
    At each step, move to the neighbour with the lowest heuristic value,
    but only if it's better than the current node. Stops at a local optimum
    or when the goal is reached.
    """
    current = start
    path = [current]
    while current != goal:
        neighbours = graph[current]
        if not neighbours:
            break

        # Pick the neighbour with the smallest heuristic (closest to goal)
        best_neighbour = min(neighbours, key=lambda n: heuristic(n, goal,
coordinates))

        current_h = heuristic(current, goal, coordinates)
        best_h = heuristic(best_neighbour, goal, coordinates)
        if best_h < current_h:
            current = best_neighbour
            path.append(current)
        else:
            # No neighbour improves on the current node -> local optimum
            break
    reached_goal = (current == goal)
    return path, reached_goal

def print_graph(graph, coordinates):
    print("Graph adjacency list:")
    for node, neighbours in graph.items():

```



```

        print(f"    {node} {coordinates[node]}: {neighbours}")
    print()
if __name__ == "__main__":
    NUM_NODES = 20
    graph, coordinates = build_graph_with_coordinates(NUM_NODES)
    print_graph(graph, coordinates)
    start_node, goal_node = "N0", "N19"
    print(f"Hill Climbing Search from {start_node} to {goal_node}:")
    path, reached_goal = hill_climbing(graph, start_node, goal_node, coordinates)
    print(" -> ".join(path))
    if reached_goal:
        print("Goal reached successfully!")
    else:
        print(f"Stuck at local optimum '{path[-1]}' -- could not reach the goal "
              f"'{goal_node}' with simple hill climbing.")

```

Output:

```

Graph adjacency list:
N0 (81, 14): ['N1', 'N15', 'N3', 'N6', 'N7']
N1 (3, 94): ['N0', 'N2', 'N4', 'N9']
N2 (35, 31): ['N1', 'N19', 'N5', 'N7']
N3 (28, 17): ['N0', 'N12', 'N17']
N4 (94, 13): ['N1', 'N13']
N5 (86, 94): ['N10', 'N11', 'N14', 'N2']
N6 (69, 11): ['N0', 'N18', 'N8']
N7 (75, 54): ['N0', 'N17', 'N2', 'N9']
N8 (4, 3): ['N12', 'N14', 'N6']
N9 (11, 27): ['N1', 'N12', 'N17', 'N7']
N10 (29, 64): ['N5']
N11 (77, 3): ['N19', 'N5']
N12 (71, 25): ['N14', 'N18', 'N3', 'N8', 'N9']
N13 (91, 83): ['N4']
N14 (89, 69): ['N12', 'N16', 'N5', 'N8']
N15 (53, 28): ['N0']
N16 (57, 75): ['N14']
N17 (35, 0): ['N3', 'N7', 'N9']
N18 (97, 20): ['N12', 'N6']
N19 (89, 54): ['N11', 'N2']

```

```
Hill Climbing Search from N0 to N19:
```

```
N0 -> N7
```

```
Stuck at local optimum 'N7' -- could not reach the goal 'N19' with simple hill
climbing.
```

Colab Link :

<https://colab.research.google.com/drive/1weOLsoNylKw1FbOqiEdCWDDhN1s3KN2j?usp=sharing>

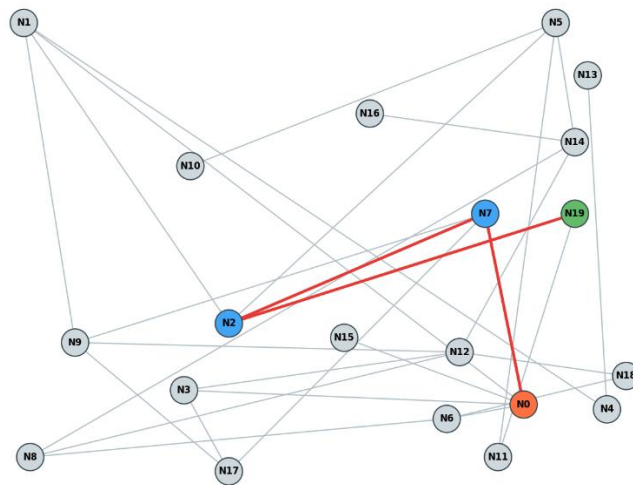
Exp No: 10	Write a Python program of beam search algorithm with 20 nodes.
26-07-2026	

Aim:

To design and implement the Beam Search algorithm for traversing a graph containing 20 nodes.

Objectives:

1. To write a Python program to implement the Beam Search algorithm.
2. To traverse a graph consisting of 20 nodes by limiting the search width.
3. To understand the trade-off between search efficiency and solution quality using Beam Search.



Program Code:

```
import random
import math

def build_graph_with_coordinates(num_nodes=20, extra_edges=15, seed=42,
grid_size=100):
    """Builds a random connected undirected graph where nodes have coordinates."""
    random.seed(seed)
    nodes = [f"N{i}" for i in range(num_nodes)]
    coordinates = {node: (random.randint(0, grid_size), random.randint(0, grid_size))
```

```

        for node in nodes}
graph = {node: [] for node in nodes}
for i in range(1, num_nodes):
    j = random.randint(0, i - 1)
    u, v = nodes[i], nodes[j]
    graph[u].append(v)
    graph[v].append(u)
for _ in range(extra_edges):
    u, v = random.sample(nodes, 2)
    if v not in graph[u]:
        graph[u].append(v)
        graph[v].append(u)
for node in graph:
    graph[node] = sorted(set(graph[node]))
return graph, coordinates

def heuristic(node, goal, coordinates):
    """Straight-line distance to the goal -- lower is better."""
    (x1, y1), (x2, y2) = coordinates[node], coordinates[goal]
    return math.hypot(x2 - x1, y2 - y1)

def beam_search(graph, start, goal, coordinates, beam_width=3, max_steps=30):
    """
    Beam Search: keeps only the `beam_width` most promising partial paths
    (by heuristic value of their last node) at every step.
    """
    # Each candidate is (path, heuristic_of_last_node)
    beam = [(start, heuristic(start, goal, coordinates))]
    visited = {start}
    for step in range(max_steps):
        candidates = []
        for path, _h in beam:
            last_node = path[-1]
            if last_node == goal:
                return path, True # Goal already reached on this path
            for neighbour in graph[last_node]:
                if neighbour not in path: # avoid cycles within a path
                    new_path = path + [neighbour]
                    candidates.append((new_path, heuristic(neighbour, goal,
coordinates)))
        if not candidates:
            break # dead end, no more candidates to expand
        # Keep only the best `beam_width` candidates

```

```

        candidates.sort(key=lambda item: item[1])
        beam = candidates[:beam_width]
        print(f" Step {step + 1}: beam = "
              f"{[(p[-1], round(h, 2)) for p, h in beam]}")
        for path, _h in beam:
            if path[-1] == goal:
                return path, True

        # Return the best partial path found if the goal wasn't reached
        best_path = min(beam, key=lambda item: item[1])[0] if beam else [start]
        return best_path, False

def print_graph(graph, coordinates):
    print("Graph adjacency list:")
    for node, neighbours in graph.items():
        print(f" {node} {coordinates[node]}: {neighbours}")
    print()

if __name__ == "__main__":
    NUM_NODES = 20
    graph, coordinates = build_graph_with_coordinates(NUM_NODES)
    print_graph(graph, coordinates)
    start_node, goal_node = "N0", "N19"
    beam_width = 3
    print(f"Beam Search from {start_node} to {goal_node} (beam width =
{beam_width}):")
    path, reached_goal = beam_search(graph, start_node, goal_node, coordinates,
beam_width)
    print()
    print(" -> ".join(path))
    if reached_goal:
        print("Goal reached successfully!")
    else:
        print(f"Goal '{goal_node}' not reached within the search limits.")

```

Output:

```

Graph adjacency list:
N0 (81, 14): ['N1', 'N15', 'N3', 'N6', 'N7']
N1 (3, 94): ['N0', 'N2', 'N4', 'N9']
N2 (35, 31): ['N1', 'N19', 'N5', 'N7']
N3 (28, 17): ['N0', 'N12', 'N17']
N4 (94, 13): ['N1', 'N13']
N5 (86, 94): ['N10', 'N11', 'N14', 'N2']
N6 (69, 11): ['N0', 'N18', 'N8']
N7 (75, 54): ['N0', 'N17', 'N2', 'N9']
N8 (4, 3): ['N12', 'N14', 'N6']

```

```
N9 (11, 27): ['N1', 'N12', 'N17', 'N7']
N10 (29, 64): ['N5']
N11 (77, 3): ['N19', 'N5']
N12 (71, 25): ['N14', 'N18', 'N3', 'N8', 'N9']
N13 (91, 83): ['N4']
N14 (89, 69): ['N12', 'N16', 'N5', 'N8']
N15 (53, 28): ['N0']
N16 (57, 75): ['N14']
N17 (35, 0): ['N3', 'N7', 'N9']
N18 (97, 20): ['N12', 'N6']
N19 (89, 54): ['N11', 'N2']
```

Beam Search from N0 to N19 (beam width = 3):

```
Step 1: beam = [('N7', 14.0), ('N15', 44.41), ('N6', 47.42)]
Step 2: beam = [('N18', 34.93), ('N2', 58.69), ('N17', 76.37)]
Step 3: beam = [('N19', 0.0), ('N12', 34.13), ('N5', 40.11)]
```

N0 -> N7 -> N2 -> N19

Goal reached successfully!

Colab Link :

<https://colab.research.google.com/drive/1VqtN8wHmhk6Hap5mUDN827T39xlnCqpt?usp=sharing>